

# Introduction to Electronic Design Automation

## Homework 2

(Problems 6-8)

Student: 吳亞倫  
ID: B13901011  
Department: 電機工程學系

### 6 Two-level Logic Minimization

#### 6. (a)

Use variable order  $abcde$ .

$$G = abce + abde + ab\bar{c}e + \bar{a}b\bar{c}e + \bar{a}\bar{b}\bar{c}\bar{e} + \bar{a}\bar{c}\bar{d}e + bc\bar{d}e + cd\bar{e}$$

$$D = \bar{a}b\bar{d}e + abcd\bar{e} + ac\bar{e}$$

Expanding them into minterms:

$$G = \{0, 1, 2, 4, 9, 11, 12, 13, 20, 25, 27, 28, 29, 31\}$$

$$D = \{9, 13, 20, 22, 28, 29, 30\}$$

Thus, we have the minterms of onset  $F$  as:

$$F = G - D = \{0, 1, 2, 4, 11, 12, 25, 27, 31\}$$

Therefore,

$$F \cup D = \{0, 1, 2, 4, 9, 11, 12, 13, 20, 22, 25, 27, 28, 29, 30, 31\}$$

$$\begin{aligned} R &= \{0, 1, \dots, 31\} - (F \cup D) \\ &= \{3, 5, 6, 7, 8, 10, 14, 15, 16, 17, 18, 19, 21, 23, 24, 26\} \end{aligned}$$

Writing them as product terms:

$$\begin{aligned} R &= \bar{a}\bar{b}\bar{c}de + \bar{a}\bar{b}c\bar{d}e + \bar{a}\bar{b}cd\bar{e} + \bar{a}\bar{b}cde + \bar{a}b\bar{c}\bar{d}\bar{e} + \bar{a}b\bar{c}d\bar{e} \\ &\quad + \bar{a}b\bar{c}d\bar{e} + \bar{a}b\bar{c}de + \bar{a}b\bar{c}\bar{d}e + \bar{a}b\bar{c}de + \bar{a}b\bar{c}\bar{d}\bar{e} + \bar{a}b\bar{c}d\bar{e} \\ &\quad + \bar{a}b\bar{c}de + \bar{a}b\bar{c}de + \bar{a}b\bar{c}\bar{d}\bar{e} + \bar{a}b\bar{c}d\bar{e} \end{aligned}$$

**6. (b)**

First, we apply the Quine–McCluskey tabular method on  $F \cup D$  to generate all prime implicants.

| Round 1                                                                                                                                                                                              | Round 2                                                                                                                                                                                                                          | Round 3                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| $\bar{a}\bar{b}\bar{c}\bar{d}\bar{e} \checkmark$                                                                                                                                                     | $\bar{a}\bar{b}\bar{c}\bar{d}$<br>$\bar{a}\bar{b}\bar{c}\bar{e}$<br>$\bar{a}\bar{b}\bar{d}\bar{e}$                                                                                                                               |                                                                   |
| $\bar{a}\bar{b}\bar{c}d\bar{e} \checkmark$<br>$\bar{a}\bar{b}c\bar{d}\bar{e} \checkmark$<br>$\bar{a}b\bar{c}\bar{d}\bar{e} \checkmark$                                                               | $\bar{a}\bar{c}\bar{d}\bar{e}$<br>$\bar{b}c\bar{d}\bar{e} \checkmark$<br>$\bar{a}c\bar{d}\bar{e} \checkmark$                                                                                                                     |                                                                   |
| $\bar{a}\bar{b}\bar{c}d\bar{e} \checkmark$<br>$\bar{a}b\bar{c}\bar{d}\bar{e} \checkmark$<br>$a\bar{b}\bar{c}\bar{d}\bar{e} \checkmark$                                                               | $\bar{a}\bar{b}\bar{c}e \checkmark$ $\bar{a}bde \checkmark$<br>$\bar{b}\bar{c}\bar{d}e \checkmark$ $\bar{a}bcd \checkmark$<br>$\bar{b}c\bar{d}\bar{e} \checkmark$ $\bar{a}b\bar{c}e \checkmark$<br>$ac\bar{d}\bar{e} \checkmark$ | $c\bar{d}\bar{e}$ $b\bar{d}e$<br>$b\bar{c}e$ $bcd$<br>$ac\bar{e}$ |
| $\bar{a}\bar{b}\bar{c}d\bar{e} \checkmark$<br>$\bar{a}b\bar{c}d\bar{e} \checkmark$<br>$a\bar{b}\bar{c}d\bar{e} \checkmark$<br>$a\bar{b}c\bar{d}\bar{e} \checkmark$<br>$abc\bar{d}\bar{e} \checkmark$ | $\bar{b}\bar{c}d\bar{e} \checkmark$ $\bar{b}c\bar{d}\bar{e} \checkmark$<br>$ac\bar{e} \checkmark$ $ab\bar{c}e \checkmark$<br>$ab\bar{d}e \checkmark$<br>$abcd \checkmark$<br>$abc\bar{e} \checkmark$                             | $abe$<br>$abc$                                                    |
| $ab\bar{c}d\bar{e} \checkmark$<br>$abc\bar{d}\bar{e} \checkmark$<br>$abcd\bar{e} \checkmark$                                                                                                         | $abde \checkmark$<br>$abce \checkmark$<br>$abcd \checkmark$                                                                                                                                                                      |                                                                   |
| $abcde \checkmark$                                                                                                                                                                                   |                                                                                                                                                                                                                                  |                                                                   |

Table 1: Quine–McCluskey tabular method for generating prime implicants

Thus, the prime implicants are

$$\{\bar{a}\bar{b}\bar{c}\bar{d}, \bar{a}\bar{b}\bar{c}\bar{e}, \bar{a}\bar{b}\bar{d}\bar{e}, \bar{a}\bar{c}\bar{d}\bar{e}, c\bar{d}\bar{e}, b\bar{d}e, \bar{b}\bar{c}e, bcd, ac\bar{e}, abe, abc\}.$$

Next, we use the Column covering of Boolean matrix to find the essential prime implicants. The Boolean matrix is as follows:

|                                             | $\bar{a}\bar{b}\bar{c}\bar{d}$ | $\bar{a}\bar{b}\bar{c}\bar{e}$ | $\bar{a}\bar{b}\bar{d}\bar{e}$ | $\bar{a}\bar{c}\bar{d}\bar{e}$ | $c\bar{d}\bar{e}$ | $b\bar{d}e$ | $\bar{b}\bar{c}e$ | $bcd$ | $ac\bar{e}$ | $abe$ | $abc$ |
|---------------------------------------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|-------------------|-------------|-------------------|-------|-------------|-------|-------|
| $m_0 = \bar{a}\bar{b}\bar{c}\bar{d}\bar{e}$ | 1                              | 1                              | 1                              | 0                              | 0                 | 0           | 0                 | 0     | 0           | 0     | 0     |
| $m_1 = \bar{a}\bar{b}\bar{c}d\bar{e}$       | 1                              | 0                              | 0                              | 1                              | 0                 | 0           | 0                 | 0     | 0           | 0     | 0     |
| $m_2 = \bar{a}\bar{b}c\bar{d}\bar{e}$       | 0                              | 1                              | 0                              | 0                              | 0                 | 0           | 0                 | 0     | 0           | 0     | 0     |
| $m_4 = \bar{a}\bar{b}cd\bar{e}$             | 0                              | 0                              | 1                              | 0                              | 1                 | 0           | 0                 | 0     | 0           | 0     | 0     |
| $m_{11} = \bar{a}b\bar{c}d\bar{e}$          | 0                              | 0                              | 0                              | 0                              | 0                 | 0           | 1                 | 0     | 0           | 0     | 0     |
| $m_{12} = \bar{a}bcd\bar{e}$                | 0                              | 0                              | 0                              | 0                              | 1                 | 0           | 0                 | 1     | 0           | 0     | 0     |
| $m_{25} = ab\bar{c}d\bar{e}$                | 0                              | 0                              | 0                              | 0                              | 0                 | 1           | 1                 | 0     | 0           | 1     | 0     |
| $m_{27} = abc\bar{d}\bar{e}$                | 0                              | 0                              | 0                              | 0                              | 0                 | 0           | 1                 | 0     | 0           | 1     | 0     |
| $m_{31} = abcde$                            | 0                              | 0                              | 0                              | 0                              | 0                 | 0           | 0                 | 0     | 0           | 1     | 1     |

Table 2: Boolean matrix

**6. (c)**

Rows  $m_2$  and  $m_{11}$  each have only one 1, so  $\bar{a}\bar{b}\bar{c}\bar{e}$  and  $b\bar{c}e$  are essential prime implicants. So we remove the rows  $m_2, m_{11}$  and the columns of  $\bar{a}\bar{b}\bar{c}\bar{e}$  and  $b\bar{c}e$ . We also remove the rows that have 1 in the removed columns, which are  $m_0, m_{25}, m_{27}$ . After removing these rows, the columns  $a\bar{c}\bar{e}$  and  $b\bar{d}e$  contain only zeros, so we remove them as well. The resulting Boolean matrix is as follows:

|                                             | $\bar{a}\bar{b}\bar{c}\bar{d}$ | $\bar{a}\bar{b}\bar{d}\bar{e}$ | $\bar{a}\bar{c}\bar{d}e$ | $c\bar{d}\bar{e}$ | $b\bar{c}\bar{d}$ | $abe$ | $abc$ |
|---------------------------------------------|--------------------------------|--------------------------------|--------------------------|-------------------|-------------------|-------|-------|
| $m_1 = \bar{a}\bar{b}\bar{c}\bar{d}e$       | 1                              | 0                              | 1                        | 0                 | 0                 | 0     | 0     |
| $m_4 = \bar{a}\bar{b}\bar{c}\bar{d}\bar{e}$ | 0                              | 1                              | 0                        | 1                 | 0                 | 0     | 0     |
| $m_{12} = \bar{a}b\bar{c}\bar{d}\bar{e}$    | 0                              | 0                              | 0                        | 1                 | 1                 | 0     | 0     |
| $m_{31} = abcde$                            | 0                              | 0                              | 0                        | 0                 | 0                 | 1     | 1     |

Table 3: Reduced Boolean matrix

**6. (d)**

Column  $c\bar{d}\bar{e}$  covers rows  $m_4$  and  $m_{12}$ . It dominates both columns  $\bar{a}\bar{b}\bar{d}\bar{e}$  and  $b\bar{c}\bar{d}$ , so we remove these two dominated columns. The resulting Boolean matrix is as follows:

|                                             | $\bar{a}\bar{b}\bar{c}\bar{d}$ | $\bar{a}\bar{c}\bar{d}e$ | $c\bar{d}\bar{e}$ | $abe$ | $abc$ |
|---------------------------------------------|--------------------------------|--------------------------|-------------------|-------|-------|
| $m_1 = \bar{a}\bar{b}\bar{c}\bar{d}e$       | 1                              | 1                        | 0                 | 0     | 0     |
| $m_4 = \bar{a}\bar{b}\bar{c}\bar{d}\bar{e}$ | 0                              | 0                        | 1                 | 0     | 0     |
| $m_{12} = \bar{a}b\bar{c}\bar{d}\bar{e}$    | 0                              | 0                        | 1                 | 0     | 0     |
| $m_{31} = abcde$                            | 0                              | 0                        | 0                 | 1     | 1     |

Table 4: Reduced Boolean matrix

Since rows  $m_4$  and  $m_{12}$  have only one 1, the prime implicant  $c\bar{d}\bar{e}$  is an induced essential prime implicant. Therefore, we select it and remove rows  $m_4$  and  $m_{12}$ .

|                                       | $\bar{a}\bar{b}\bar{c}\bar{d}$ | $\bar{a}\bar{c}\bar{d}e$ | $abe$ | $abc$ |
|---------------------------------------|--------------------------------|--------------------------|-------|-------|
| $m_1 = \bar{a}\bar{b}\bar{c}\bar{d}e$ | 1                              | 1                        | 0     | 0     |
| $m_{31} = abcde$                      | 0                              | 0                        | 1     | 1     |

Table 5: Reduced Boolean matrix

**6. (e)**

After the reductions in (d), there is no cyclic core to solve, since the remaining matrix is decomposed into two independent single-row choices:

$$m_1 : \{\bar{a}\bar{b}\bar{c}\bar{d}, \bar{a}\bar{c}\bar{d}e\}$$

$$m_{31} : \{abe, abc\}$$

Thus, the independent set heuristic is not applicable. We only need to choose one prime implicant from each set. For example, we choose

$$\bar{a}\bar{b}\bar{c}\bar{d} \quad \text{and} \quad abe.$$

Therefore, the selected prime implicants are

$$\bar{a}\bar{b}\bar{c}\bar{e}, \quad b\bar{c}e, \quad c\bar{d}\bar{e}, \quad \bar{a}\bar{b}\bar{c}\bar{d}, \quad abe.$$

**6. (f)**

There are four minimum covers with 5 prime implicants:

$$G_1^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{b}\bar{c}\bar{d} + abe.$$

$$G_2^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{c}\bar{d}e + abe.$$

$$G_3^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{b}\bar{c}\bar{d} + abc.$$

$$G_4^* = \bar{a}\bar{b}\bar{c}\bar{e} + b\bar{c}e + c\bar{d}\bar{e} + \bar{a}\bar{c}\bar{d}e + abc.$$

**7 Static Timing Analysis****7. (a)**

In this part, we construct the circuit as a DAG, where each node  $v \in V$  represents a gate and each edge  $e \in E \subseteq V \times V$  represents a connection between gates. Let  $R(v)$  denote the required time at the input of node  $v$ ,  $A(v)$  denote the arrival time at the output of node  $v$ , and  $d_{u,v}$  denotes the delay from node  $u$ 's output to node  $v$ 's output. Also, we define the slack of node  $v$  as

$$S(v) = R(v) - A(v).$$

Suppose there is a node  $v$  such that

$$S(v) = R(v) - A(v) \leq c$$

We want to show that there exists a path satisfying every node on the path has slack at most  $c$  from some primary input to  $v$  and from  $v$  to some primary output.

**1. From some primary input to  $v$** 

First, if  $v$  is a primary input, then we are done. Otherwise, since

$$A(v) = \max_{u:(u,v) \in E} (A(u) + d_{u,v})$$

there exist a node  $u$  such that there is an edge from  $u$  to  $v$  and

$$A(v) = A(u) + d_{u,v}.$$

And since

$$R(u) = \min_{w:(u,w) \in E} (R(w) - d_{u,w}) \leq R(v) - d_{u,v},$$

we have

$$S(u) = R(u) - A(u) \leq R(v) - d_{u,v} - A(u) = R(v) - A(v) = S(v) \leq c.$$

Thus, we can apply the same argument on  $u$  and repeat this process until we reach a primary input.

**2. From  $v$  to some primary output**

First, if  $v$  is a primary output, then we are done. Otherwise, since

$$R(v) = \min_{w:(v,w) \in E} (R(w) - d_{v,w})$$

there exist a node  $w$  such that there is an edge from  $v$  to  $w$  and

$$R(v) = R(w) - d_{v,w}.$$

And since

$$A(w) = \max_{x:(x,w) \in E} (A(x) + d_{x,w}) \geq A(v) + d_{v,w},$$

we have

$$S(w) = R(w) - A(w) \leq R(v) + d_{v,w} - A(v) - d_{v,w} = R(v) - A(v) = S(v) \leq c.$$

Thus, we can apply the same argument on  $w$  and repeat this process until we reach a primary output.

Hence, there exists a path from some primary input to  $v$  and from  $v$  to some primary output such that the slack of each node on these paths is at most  $c$ .

## 7. (b)

The equivalence DAG of the given circuit is as Figure 1 below. The delay of each gate is labeled on the corresponding node. Arrival time, required time and slack of each node are labeled on the corresponding edge in the format of  $(a, r, s)$ , where  $a$  is the arrival time,  $r$  is the required time and  $s$  is the slack. The critical path is highlighted in red.

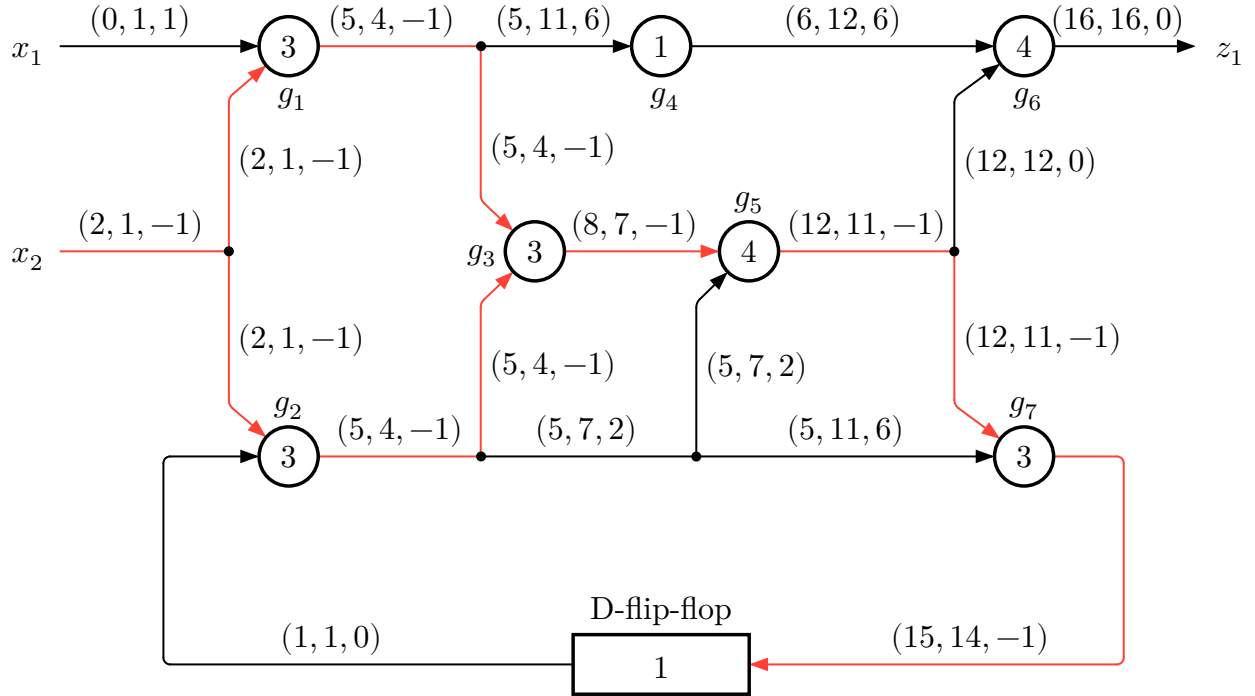


Figure 1: static timing analysis result

## 8 Technology Mapping

In this problem, I wrote two versions of the program. The first one is the basic DAGON mapping algorithm. The second one is an improved version, where I allow adding a pair of inverters for each wire in the subject graph and adding a pattern of a wire that matches two inverters with zero cost.

The node numbering I used in my input file is shown in Figure 2.

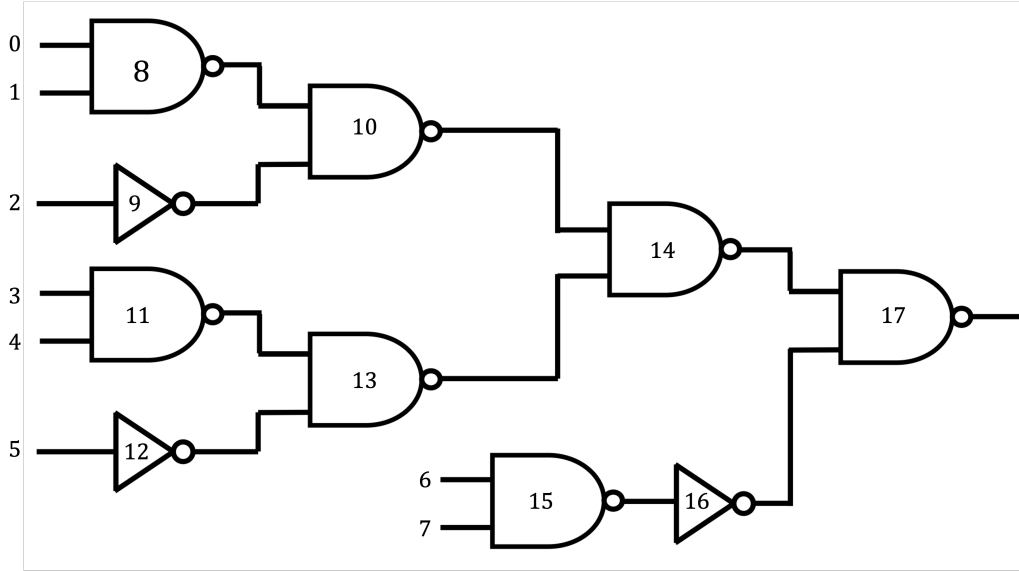


Figure 2: Node numbering used in `input.txt`

First, for the basic version without improvement, the result is:

| Node | Minimum cost | Selected cell |
|------|--------------|---------------|
| 8    | 2            | nand2         |
| 9    | 1            | inv           |
| 10   | 5            | nand2         |
| 11   | 2            | nand2         |
| 12   | 1            | inv           |
| 13   | 5            | nand2         |
| 14   | 12           | nand2         |
| 15   | 2            | nand2         |
| 16   | 3            | inv           |
| 17   | 15           | nand3         |

Table 6: Optimal cost and selected cell of each subject node

Thus, the final cover without improvement is obtained by tracing back from output node 17:

{8 : nand2, 9 : inv, 10 : nand2, 11 : nand2, 12 : inv, 13 : nand2, 14 : nand2, 17 : nand3}.

The nand3 at node 17 covers nodes 15, 16, and 17 together. Therefore, the minimum total area is

$$2 + 1 + 2 + 2 + 1 + 2 + 2 + 3 = 15.$$

Next, for the improved version, the output on original nodes is:

| Node | Minimum cost | Selected cell |
|------|--------------|---------------|
| 8    | 2            | nand2         |
| 9    | 1            | inv           |
| 10   | 5            | nand2         |
| 11   | 2            | nand2         |
| 12   | 1            | inv           |
| 13   | 5            | nand2         |
| 14   | 10           | nand2         |
| 15   | 2            | nand2         |
| 16   | 3            | inv           |
| 17   | 12           | oai21         |

Table 7: Improved result on original subject nodes. Only contains cost and cell at the original gate outputs, without the inserted inverters. For more details, see `output_improve.txt` in Appendix.

The minimum total area becomes 12. This is smaller than 15 because the inserted inverter pairs create extra matching choices. In particular, node 17 can be matched by `oai21` in the improved graph, so the final cost is reduced.

However, if we only apply the original DAGON algorithm directly on the given subject graph, the answer is still 15.

The c++ code of both versions and the input/output files are included in the appendix.

## Appendix: `daigon.cpp`

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4 #define pb push_back
5 #define endl '\n'
6 #define AI(x) begin(x),end(x)
7
8
9 const int INF = 1e9;
10
11 enum class GateType {
12     PI,
13     INV,
14     NAND,
15     NOR,
16 };
17
18 string toUpper(string s) {
19     for (char &c : s) c = toupper(c);
20     return s;
21 }
22
23 GateType s2gate_type(string s) {
24     s = toUpper(s);
25
26     if (s == "PI" || s == "IN") return GateType::PI;
27     if (s == "INV") return GateType::INV;
28     if (s == "NAND") return GateType::NAND;
29     if (s == "NOR") return GateType::NOR;
30
31     cerr << "Unknown gate type: " << s << endl;
32     exit(1);
33 }
34
35 bool isCommutative(GateType t) {
36     return t == GateType::NAND || t == GateType::NOR;

```

cpp

```

37 }
38
39 struct node {
40     int id;
41     GateType type;
42     vector<int> fanin;
43     vector<int> fanout;
44
45     node() {}
46     node(int i, GateType g) : id(i), type(g) {}
47 };
48
49 vector<int> topological_sort(vector<node>& g) {
50     int n = g.size();
51     vector<int> indeg(n), order;
52     queue<int> q;
53
54     for (int i = 0; i < n; i++) {
55         indeg[i] = g[i].fanin.size();
56         if (indeg[i] == 0) q.push(i);
57     }
58
59     while (!q.empty()) {
60         int u = q.front();
61         q.pop();
62
63         order.pb(u);
64
65         for (int v : g[u].fanout) {
66             indeg[v]--;
67             if (indeg[v] == 0) q.push(v);
68         }
69     }
70
71     return order;
72 }
73
74 // 回傳 pattern subtree match 到 subject subtree 後, boundary dp cost 總和
75 // 若不能 match, 回傳 INF
76 int matchCost(int sid, int pid,
77               vector<node>& sg,
78               vector<node>& pg,
79               vector<int>& dp) {
80
81     node& s = sg[sid];
82     node& p = pg[pid];
83
84     if (p.type == GateType::PI) {
85         return dp[sid];
86     }
87
88     if (s.type != p.type) return INF;
89     if (s.fanin.size() != p.fanin.size()) return INF;
90
91     int n = s.fanin.size();
92
93     vector<int> idx(n);
94     iota(idx.begin(), idx.end(), 0);
95
96     int best = INF;
97
98     do {
99         int total = 0;
100         bool ok = true;
101
102         for (int i = 0; i < n; i++) {
103             int s_child = s.fanin[idx[i]];
104             int p_child = p.fanin[i];
105
106             int sub = matchCost(s_child, p_child, sg, pg, dp);
107
108             if (sub >= INF) {
109                 ok = false;
110                 break;
111             }
112
113             total += sub;
114         }
115
116         if (ok) best = min(best, total);
117     } while (isCommutative(s.type) &&
118             next_permutation(idx.begin(), idx.end()));
119
120     return best;
121 }
122
123
124 int32_t main() {
125     ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
126     int ns, ms;

```

```

127     cin >> ns >> ms;
128
129     vector<node> sg(ns);
130
131     for (int i = 0; i < ns; i++) {
132         string t;
133         cin >> t;
134         sg[i] = node(i, s2gate_type(t));
135     }
136
137     for (int i = 0; i < ms; i++) {
138         int a, b;
139         cin >> a >> b;
140
141         sg[a].fanout.pb(b);
142         sg[b].fanin.pb(a);
143     }
144
145     vector<int> subject_order = topological_sort(sg);
146
147     int p;
148     cin >> p;
149
150     vector<vector<node>> patterns(p);
151     vector<int> cost(p);
152     vector<string> name(p);
153     vector<int> root(p);
154
155     for (int i = 0; i < p; i++) {
156         cin >> name[i] >> cost[i];
157
158         int n, m, r;
159         cin >> n >> m >> r;
160
161         root[i] = r;
162         patterns[i].resize(n);
163
164         for (int j = 0; j < n; j++) {
165             string t;
166             cin >> t;
167             patterns[i][j] = node(j, s2gate_type(t));
168         }
169
170         for (int j = 0; j < m; j++) {
171             int a, b;
172             cin >> a >> b;
173
174             patterns[i][a].fanout.pb(b);
175             patterns[i][b].fanin.pb(a);
176         }
177     }
178
179     vector<int> dp(ns, INF);
180     vector<int> choose(ns, -1);
181
182     for (int v : subject_order) {
183         if (sg[v].type == GateType::PI) {
184             dp[v] = 0;
185             continue;
186         }
187
188         for (int i = 0; i < p; i++) {
189             int boundary_cost = matchCost(v, root[i], sg, patterns[i], dp);
190
191             if (boundary_cost >= INF) continue;
192
193             int total_cost = cost[i] + boundary_cost;
194
195             if (total_cost < dp[v]) {
196                 dp[v] = total_cost;
197                 choose[v] = i;
198             }
199         }
200     }
201
202     // 預設所有 fanout empty 的 node 都是 output
203     int ans = 0;
204
205     for (int i = 0; i < ns; i++) {
206         if (sg[i].fanout.empty()) {
207             ans += dp[i];
208         }
209     }
210
211     cout << "Minimum total cost = " << ans << endl;
212
213     for (int i = 0; i < ns; i++) {
214         cout << "node " << i << ": ";
215
216         if (sg[i].fanin.empty()) {

```

```

217     cout << "PI, cost = 0" << endl;
218 } else {
219     cout << "cost = " << dp[i];
220
221     if (choose[i] != -1) {
222         cout << ", cell = " << name[choose[i]];
223     } else {
224         cout << ", cell = NONE";
225     }
226
227     cout << endl;
228 }
229 }
230
231 return 0;
232 }
233

```

## Appendix: dagon\_improve.cpp

cpp

```

1 // dagon_improve.cpp
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 #define ll long long
6 #define pb push_back
7 #define endl '\n'
8
9 const ll INF = (ll)4e18;
10
11 enum class GateType {
12     PI,
13     INV,
14     NAND,
15     NOR,
16 };
17
18 string toUpper(string s) {
19     for (char &c : s) c = toupper(c);
20     return s;
21 }
22
23 GateType s2gate_type(string s) {
24     s = toUpper(s);
25
26     if (s == "PI" || s == "IN") return GateType::PI;
27     if (s == "INV") return GateType::INV;
28     if (s == "NAND") return GateType::NAND;
29     if (s == "NOR") return GateType::NOR;
30
31     cerr << "Unknown gate type: " << s << endl;
32     exit(1);
33 }
34
35 string gate2str(GateType t) {
36     if (t == GateType::PI) return "PI";
37     if (t == GateType::INV) return "INV";
38     if (t == GateType::NAND) return "NAND";
39     if (t == GateType::NOR) return "NOR";
40     return "?";
41 }
42
43 bool isCommutative(GateType t) {
44     return t == GateType::NAND || t == GateType::NOR;
45 }
46
47 struct Node {
48     int id;
49     GateType type;
50     vector<int> fanin;
51     vector<int> fanout;
52
53     Node() {}
54     Node(int i, GateType g) : id(i), type(g) {}
55 };
56
57 struct Pattern {
58     string name;
59     ll cost;
60     int root;
61     vector<Node> g;
62 };
63
64 void addEdge(vector<Node> &g, int u, int v) {
65     g[u].fanout.pb(v);
66     g[v].fanin.pb(u);
67 }
68

```

```

69 vector<int> topological_sort(vector<Node> &g) {
70     int n = (int)g.size();
71     vector<int> indeg(n), order;
72     queue<int> q;
73
74     for (int i = 0; i < n; i++) {
75         indeg[i] = (int)g[i].fanin.size();
76         if (indeg[i] == 0) q.push(i);
77     }
78
79     while (!q.empty()) {
80         int u = q.front();
81         q.pop();
82
83         order.pb(u);
84
85         for (int v : g[u].fanout) {
86             if (--indeg[v] == 0) q.push(v);
87         }
88     }
89
90     if ((int)order.size() != n) {
91         cerr << "Error: graph is not a DAG." << endl;
92         exit(1);
93     }
94
95     return order;
96 }
97
98 // 對 subject graph 每條原始 edge u -> v 插入兩個 INV :
99 // u -> inv1 -> inv2 -> v
100 void insertInverterPairs(vector<Node> &sg,
101                          const vector<pair<int,int>> &edges,
102                          vector<char> &isInserted) {
103     for (auto &x : sg) {
104         x.fanin.clear();
105         x.fanout.clear();
106     }
107
108     for (auto [u, v] : edges) {
109         int inv1 = (int)sg.size();
110         sg.pb(Node(inv1, GateType::INV));
111         isInserted.pb(true);
112
113         int inv2 = (int)sg.size();
114         sg.pb(Node(inv2, GateType::INV));
115         isInserted.pb(true);
116
117         addEdge(sg, u, inv1);
118         addEdge(sg, inv1, inv2);
119         addEdge(sg, inv2, v);
120     }
121 }
122
123 // 回傳 pattern subtree match 到 subject subtree 後的 boundary dp cost 總和。
124 // 若不能 match, 回傳 INF.
125 ll matchCost(int sid, int pid,
126              vector<Node> &sg,
127              vector<Node> &pg,
128              vector<ll> &dp,
129              vector<char> &isInserted) {
130
131     Node &s = sg[sid];
132     Node &pnode = pg[pid];
133
134     // pattern 的 PI 是 boundary, 可以停在任何 subject node
135     if (pnode.type == GateType::PI) {
136         return dp[sid];
137     }
138
139     ll bestSkip = INF;
140
141     // 嘗試跳過 virtual INV pair:
142     // u -> inv1 -> inv2
143     // sid = inv2 時, 可以直接嘗試 match u
144     if (isInserted[sid] &&
145         s.type == GateType::INV &&
146         s.fanin.size() == 1) {
147
148         int inv1 = s.fanin[0];
149
150         if (isInserted[inv1] &&
151             sg[inv1].type == GateType::INV &&
152             sg[inv1].fanin.size() == 1) {
153
154             int u = sg[inv1].fanin[0];
155
156             bestSkip = matchCost(u, pid, sg, pg, dp, isInserted);
157         }
158     }

```

```

159
160     if (s.type != pnode.type) return bestSkip;
161     if (s.fanin.size() != pnode.fanin.size()) return bestSkip;
162
163     int n = (int)s.fanin.size();
164
165     vector<int> idx(n);
166     iota(idx.begin(), idx.end(), 0);
167
168     ll best = INF;
169
170     do {
171         ll total = 0;
172         bool ok = true;
173
174         for (int i = 0; i < n; i++) {
175             int s_child = s.fanin[idx[i]];
176             int p_child = pnode.fanin[i];
177
178             ll sub = matchCost(s_child, p_child, sg, pg, dp, isInserted);
179
180             if (sub >= INF) {
181                 ok = false;
182                 break;
183             }
184
185             total += sub;
186
187             if (total >= INF) {
188                 ok = false;
189                 break;
190             }
191         }
192
193         if (ok) best = min(best, total);
194     } while (isCommutative(s.type) &&
195             next_permutation(idx.begin(), idx.end()));
196
197     return min(best, bestSkip);
198 }
199
200 int32_t main() {
201     ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
202     int ns, ms;
203     cin >> ns >> ms;
204
205     vector<Node> sg(ns);
206
207     for (int i = 0; i < ns; i++) {
208         string t;
209         cin >> t;
210         sg[i] = Node(i, s2gate_type(t));
211     }
212
213     vector<pair<int,int>> original_edges;
214
215     for (int i = 0; i < ms; i++) {
216         int a, b;
217         cin >> a >> b;
218         original_edges.pb({a, b});
219         addEdge(sg, a, b);
220     }
221
222     // 記錄原始 output, 因為後面會清除並重建 edge
223     vector<int> original_outputs;
224     for (int i = 0; i < ns; i++) {
225         if (sg[i].fanout.empty()) original_outputs.pb(i);
226     }
227
228     // improve DAGON: 每條 wire 插入兩個 INV
229     vector<char> isInserted(ns, false);
230     insertInverterPairs(sg, original_edges, isInserted);
231
232     int p;
233     cin >> p;
234
235     vector<Pattern> patterns;
236
237     for (int i = 0; i < p; i++) {
238         Pattern pat;
239         cin >> pat.name >> pat.cost;
240
241         int n, m, r;
242         cin >> n >> m >> r;
243
244         pat.root = r;
245         pat.g.resize(n);
246
247         for (int j = 0; j < n; j++) {

```

```

249     string t;
250     cin >> t;
251     pat.g[j] = Node(j, s2gate_type(t));
252 }
253
254 for (int j = 0; j < m; j++) {
255     int a, b;
256     cin >> a >> b;
257     addEdge(pat.g, a, b);
258 }
259
260 patterns.pb(pat);
261 }
262
263 vector<int> order = topological_sort(sg);
264
265 int nsg = (int)sg.size();
266 vector<ll> dp(nsg, INF);
267 vector<int> choose(nsg, -1);
268
269 for (int v : order) {
270     if (sg[v].type == GateType::PI) {
271         dp[v] = 0;
272         continue;
273     }
274
275     for (int i = 0; i < (int)patterns.size(); i++) {
276         ll boundary_cost = matchCost(v, patterns[i].root, sg, patterns[i].g, dp, isInserted);
277
278         if (boundary_cost >= INF) continue;
279
280         ll total_cost = patterns[i].cost + boundary_cost;
281
282         if (total_cost < dp[v]) {
283             dp[v] = total_cost;
284             choose[v] = i;
285         }
286     }
287 }
288
289 ll ans = 0;
290 for (int out : original_outputs) {
291     ans += dp[out];
292 }
293
294 cout << "Minimum total cost = " << ans << endl;
295
296 cout << endl;
297 cout << "[Original nodes]" << endl;
298 for (int i = 0; i < ns; i++) {
299     cout << "node " << i << " (" << gate2str(sg[i].type) << "): ";
300
301     if (sg[i].type == GateType::PI) {
302         cout << "PI, cost = 0";
303     } else {
304         cout << "cost = " << dp[i];
305
306         if (choose[i] != -1) {
307             cout << ", cell = " << patterns[choose[i]].name;
308         } else {
309             cout << ", cell = NONE";
310         }
311     }
312
313     bool is_output = false;
314     for (int out : original_outputs) {
315         if (out == i) is_output = true;
316     }
317     if (is_output) cout << " [OUTPUT]";
318
319     cout << endl;
320 }
321
322 cout << endl;
323 cout << "[Inserted inverter-pair nodes]" << endl;
324
325 for (int i = ns; i < nsg; i++) {
326     cout << "node " << i << " (" << gate2str(sg[i].type) << "): ";
327     cout << "cost = " << dp[i];
328
329     if (choose[i] != -1) {
330         cout << ", cell = " << patterns[choose[i]].name;
331     } else {
332         cout << ", cell = NONE";
333     }
334
335     cout << endl;
336 }
337
338 return 0;

```

```
339 }
340
```

## Appendix: input.txt

txt

```
18 17
PI PI PI PI PI PI PI PI NAND INV NAND INV NAND INV NAND INV NAND
0 8
1 8
2 9
8 10
9 10
3 11
4 11
5 12
11 13
12 13
10 14
13 14
6 15
7 15
15 16
14 17
16 17
10
inv 1
2 1 1
PI INV
0 1
buf 0
3 2 2
PI INV INV
0 1
1 2
nand2 2
3 2 2
PI PI NAND
0 2
1 2
and2 3
4 3 3
PI PI NAND INV
0 2
1 2
2 3
nand3 3
6 5 5
PI PI PI NAND INV NAND
0 3
1 3
3 4
4 5
2 5
or2 4
5 4 4
PI PI INV INV NAND
0 2
1 3
2 4
3 4
aoi21 3
7 6 6
PI PI PI NAND INV NAND INV
0 3
1 3
2 4
3 5
4 5
5 6
oai21 3
7 6 6
PI PI PI INV INV NAND NAND
0 3
1 4
3 5
4 5
5 6
2 6
aoi22 4
8 7 7
PI PI PI PI NAND NAND NAND INV
0 4
1 4
2 5
3 5
4 6
5 6
6 7
nand4 4
9 8 8
PI PI PI PI NAND INV NAND INV NAND
0 4
1 4
4 5
5 6
```

```

2 6
6 7
7 8
3 8

```

## Appendix: output.txt

txt

```

Minimum total cost = 15
node 0: PI, cost = 0
node 1: PI, cost = 0
node 2: PI, cost = 0
node 3: PI, cost = 0
node 4: PI, cost = 0
node 5: PI, cost = 0
node 6: PI, cost = 0
node 7: PI, cost = 0
node 8: cost = 2, cell = nand2
node 9: cost = 1, cell = inv
node 10: cost = 5, cell = nand2
node 11: cost = 2, cell = nand2
node 12: cost = 1, cell = inv
node 13: cost = 5, cell = nand2
node 14: cost = 12, cell = nand2
node 15: cost = 2, cell = nand2
node 16: cost = 3, cell = inv
node 17: cost = 15, cell = nand3

```

## Appendix: output\_improve.txt

txt

```

Minimum total cost = 12

[Original nodes]
node 0 (PI): PI, cost = 0
node 1 (PI): PI, cost = 0
node 2 (PI): PI, cost = 0
node 3 (PI): PI, cost = 0
node 4 (PI): PI, cost = 0
node 5 (PI): PI, cost = 0
node 6 (PI): PI, cost = 0
node 7 (PI): PI, cost = 0
node 8 (NAND): cost = 2, cell = nand2
node 9 (INV): cost = 1, cell = inv
node 10 (NAND): cost = 5, cell = nand2
node 11 (NAND): cost = 2, cell = nand2
node 12 (INV): cost = 1, cell = inv
node 13 (NAND): cost = 5, cell = nand2
node 14 (NAND): cost = 10, cell = nand2
node 15 (NAND): cost = 2, cell = nand2
node 16 (INV): cost = 3, cell = inv
node 17 (NAND): cost = 12, cell = oai21 [OUTPUT]

[Inserted inverter-pair nodes]
node 18 (INV): cost = 1, cell = inv
node 19 (INV): cost = 0, cell = buf
node 20 (INV): cost = 1, cell = inv
node 21 (INV): cost = 0, cell = buf
node 22 (INV): cost = 1, cell = inv
node 23 (INV): cost = 0, cell = buf
node 24 (INV): cost = 3, cell = inv
node 25 (INV): cost = 2, cell = buf
node 26 (INV): cost = 0, cell = buf
node 27 (INV): cost = 1, cell = inv
node 28 (INV): cost = 1, cell = inv
node 29 (INV): cost = 0, cell = buf
node 30 (INV): cost = 1, cell = inv
node 31 (INV): cost = 0, cell = buf
node 32 (INV): cost = 1, cell = inv
node 33 (INV): cost = 0, cell = buf
node 34 (INV): cost = 3, cell = inv
node 35 (INV): cost = 2, cell = buf
node 36 (INV): cost = 0, cell = buf
node 37 (INV): cost = 1, cell = inv
node 38 (INV): cost = 3, cell = aoi21
node 39 (INV): cost = 4, cell = inv
node 40 (INV): cost = 3, cell = aoi21
node 41 (INV): cost = 4, cell = inv
node 42 (INV): cost = 1, cell = inv
node 43 (INV): cost = 0, cell = buf
node 44 (INV): cost = 1, cell = inv
node 45 (INV): cost = 0, cell = buf
node 46 (INV): cost = 3, cell = inv

```

```
node 47 (INV): cost = 2, cell = buf
node 48 (INV): cost = 9, cell = aoi21
node 49 (INV): cost = 10, cell = inv
node 50 (INV): cost = 2, cell = buf
node 51 (INV): cost = 3, cell = inv
```